MazeRunner 3D: Technology Stack & System Architecture

Brief Technical Reference: Engine Components, Library Layers & Pipeline Execution

Project: MazeRunner 3D Simulation	**Course: Macro Project-I (Subject Code: 15252110)**
Institution: MITS Gwalior (CSE III Semester)	**Candidate: Niyati Patel (Roll: BTCS25O1093)**
Language / Toolchain: C++17 • MinGW-w64 UCRT64	**Supervisor: Mrs. Manisha Pathak (Assistant Professor)**

1. Technology Stack Breakdown

MazeRunner is built from scratch in modern C++17 with zero third-party game engine dependencies. The project leverages industry-standard low-level libraries for hardware-accelerated 3D rendering, windowing, linear algebra, and embedded transactional storage:

Component / Layer	Library / API	Version	Role & Functional Purpose in MazeRunner
Programming Language	ISO C++ (C++17)	GCC 15.2 / C++17	Core simulation logic, deterministic memory management, RAII, STL algorithms.
Graphics API	OpenGL Core Profile	3.3 Core	GPU rendering pipeline, GLSL programmable shaders, VBO/VAO instancing, depth testing.
Window & Input	GLFW	3.4.0	Cross-platform window creation, OpenGL context initialization, keyboard/mouse event dispatch.
OpenGL Loader	GLEW	2.2.0	Extension wrangler; binds modern OpenGL 3.3 function pointers at application launch.
Mathematics Library	GLM	1.0.3 (Header-only)	3D vector/matrix transformations: Perspective projection, View lookAt, Euler rotations.
Database Engine	SQLite3	3.52.0 (C-API)	Embedded ACID storage: 3 save slots, session analytics, high-score leaderboards (WAL mode).
Build & Automation	CMake & Ninja	CMake 4.3 / Ninja 1.13	Cross-platform build system generation and rapid parallel compilation.

2. System Architecture & Modular Organization

The codebase is divided into 5 modular, decoupled architectural layers ensuring separation of concerns:

Subsystem Layer	Primary Modules	Architectural Responsibility
Presentation & Rendering	`graphics.h / cpp`	Compiles GLSL shaders, streams VBOs, executes Blinn-Phong lighting, renders 3D world, keys, exit portal, and 2D HUD minimap.
Game Logic & Physics	`game.h / cpp` `player.h / cpp`	Updates game state (Menu, Playing, Over, Victory), handles 6-DOF camera, executes smooth AABB wall-sliding collision, and manages inventory keys.
Autonomous AI & Navigation	`monster.h / cpp` `pathfinding.h / cpp`	Controls predator entities via a 4-state Finite State Machine (Patrol, Detect, Chase, Search) with line-of-sight raycasting and dynamic A* chasing.
Procedural Topology	`maze.h / cpp`	Generates perfect mazes via DFS, Prim's, or Wilson's algorithms; validates spanning-tree connectivity; converts 2D bitmasks to 3D block grids.
Persistence & Telemetry	`database.h / cpp`	Manages SQLite connection in WAL mode; executes multi-slot save/load serialization; queries real-time leaderboards via SQL views.

3. How It Works: Core Workflows & Execution Pipelines

The following pipelines govern the data flow and runtime execution of the MazeRunner system:

A. Procedural Generation to 3D World Transformation:

1. *Generation:* The selected algorithm (DFS, Prim, or Wilson) operates on an N × M grid, setting 4-bit wall flags per cell.
2. *Validation:* Breadth-First Search (BFS) verifies reachability from (0,0) and confirms the passage count equals N×M - 1 (guaranteeing a loopless spanning tree).
3. *Block Synthesis:* toBlockGrid() expands the logical cells into an odd-dimension coordinate grid (2H+1) × (2W+1) where '#' denotes physical wall blocks and ' ' denotes open corridors.
4. *GPU Upload:* The renderer iterates the block grid, generating 3D translation matrices for each wall unit and submitting them to vertex buffers.

B. Real-Time 3D Rendering & Illumination Pipeline:

1. *Camera View Matrix:* Formed using glm::lookAt(position, position + front, up) with Euler yaw and pitch angles.
2. *Vertex Processing:* GLSL vertex shaders transform local cube vertices via Model-View-Projection (MVP) matrices.
3. *Blinn-Phong Spotlight:* Fragment shaders compute diffuse and specular reflections originating from the player's camera position along the forward view vector, modulated by smooth inner (14°) and outer (22°) cone cut-offs.
4. *Distance Fog:* Exponential decay fog (exp(-dist * 0.045)) blends corridor geometry into the dark background, establishing atmospheric depth.
5. *2D HUD Overlay:* An orthographic projection matrix renders screen-space quads for the health bar, stamina bar, key badges, crosshair, and top-right minimap with real-time player and monster markers.

C. Autonomous Monster AI & Dynamic Pathfinding:

1. *Patrol State:* The monster wanders between local corridor waypoints at 2.0 m/s.
2. *Line-of-Sight (LOS) Detection:* A raycast traverser checks unobstructed grid sightlines up to 10 units within a 110° forward field-of-view cone.
3. *Dynamic Chase:* Upon visual contact, the monster enters CHASE mode, accelerating to 4.5 m/s. The engine recalculates optimal A* paths to the player every 400ms.
4. *Search & Cooldown:* When the player breaks line of sight behind a corner, the monster transitions to SEARCH, investigating the last-known position for 2.5s before resuming patrol.
5. *Corner Clearance:* Collision bounding checks ensure monsters slide around corridor corners without clipping.

D. Database Persistence & Save/Load Pipeline:

1. *Write-Ahead Logging (WAL):* PRAGMA journal_mode=WAL; enables concurrent, zero-stutter reads and writes.
2. *Multi-Slot Save Game:* Serializes player coordinates, orientation, health, stamina, collected keys bitmask, elapsed time, and maze seed into save_slots with atomic transactions.
3. *Analytics & Leaderboards:* Completed game runs insert session records into sessions, and the view v_leaderboard computes player rankings ordered by score and completion time.

4. Quick Reference: Application Launch Modes

Command Line Execution	Description & Behavior	Output / Interface
`.\mazerunner.exe`	Interactive main menu: launch 3D game, test suite, leaderboard, or terminal solver.	**Console Interactive Menu**
`.\mazerunner.exe --3d`	Directly launches hardware-accelerated 3D OpenGL first-person exploration game.	**OpenGL 3D Window (1280x720)**
`.\mazerunner.exe --test`	Executes the full 19-test automated verification suite for Milestones 1 through 5.	**100% Automated Test Report**
`.\mazerunner.exe --benchmark`	Benchmarks runtime and node expansions for DFS, Prim, Wilson, BFS, A*, and Bi-BFS.	**Performance Matrix Table**
`.\mazerunner.exe --db`	Queries SQLite embedded database and displays top-ranking player leaderboards.	**Formatted SQL Leaderboard**